

STOCK PRICE PREDICTION PROJECT

Using LSTM Deep Learning Model

Submitted by: Sourav Garu, Harsh Raj

Trainer: Debasish Sahoo

Department of Electrical & Electronics Engineering

Haldia Institute of Technology

3rd Year – 6th Semester

Academic Year: 2025–26

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our trainer Debasish Sahoo for his continuous guidance, support, and motivation throughout the development of this project. We are also thankful to Haldia Institute of Technology for providing the platform and resources required to successfully complete this work. Special thanks to our teammates and friends for their valuable suggestions and encouragement during implementation and testing.

ABSTRACT

Stock market prediction is a challenging task due to the volatile and nonlinear nature of financial data. This project uses a Long Short-Term Memory (LSTM) neural network to predict future stock prices based on historical stock data. The model is trained in Google Colab and integrated into a full-stack web application consisting of a FastAPI backend and an HTML, CSS, and JavaScript frontend.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement & Objectives
3. System Architecture
4. LSTM Model Training (Google Colab)
5. Backend Development (FastAPI)
6. Frontend Development (HTML, JavaScript, CSS)
7. Outputs Results
8. Conclusion

1.INTRODUCTION

Stock market prediction is an important application of machine learning and data analysis. Due to the highly dynamic and complex nature of financial markets, accurately forecasting stock prices is a challenging task. Traditional statistical methods often fail to capture non-linear patterns present in stock data. Therefore, deep learning techniques such as Long Short-Term Memory (LSTM) networks are widely used for time-series prediction because they can learn long-term dependencies from sequential data. This project presents a stock price prediction system using an LSTM model trained on historical stock market data collected using the yFinance library and trained in Google Colab. The trained model predicts the next day's closing price based on the previous 30 days of stock prices. The system is implemented as a full-stack web application with a FastAPI backend and an HTML, CSS, and JavaScript frontend that allows users to enter a stock symbol and view prediction results along with graphical visualization of actual versus predicted prices. This project demonstrates the practical application of deep learning in financial forecasting integrated with modern web technologies.

2. PROBLEM STATEMENT & OBJECTIVES

Problem Statement: To design and implement a system that predicts stock prices using historical data and provides predictions through a web application.

- Train LSTM model on historical stock data.
- Build FastAPI backend for prediction serving.
- Create frontend UI using HTML, CSS, JavaScript.
- Show last 30 days data and actual vs predicted graph.

3. SYSTEM ARCHITECTURE

Workflow: User → Frontend (HTML/CSS/JS) → FastAPI Backend → LSTM Model → Prediction → Visualization.

4. LSTM MODEL TRAINING (GOOGLE COLAB)

```
!pip install yfinance

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

stock = "GOOG"
data = yf.download(stock, start="2015-01-01", end="2025-01-01")
data.reset_index(inplace=True)

data.head()
plt.figure(figsize=(10,5))
plt.plot(data['Close'], label='Closing Price')
plt.title("Original Stock Price")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```

close_data = data[['Close']]

scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(close_data)

X = []
y = []

for i in range(60, len(scaled_data)):
    X.append(scaled_data[i-60:i])
    y.append(scaled_data[i])

X, y = np.array(X), np.array(y)

split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(60,1)))
model.add(Dropout(0.2))

model.add(LSTM(50))
model.add(Dropout(0.2))

model.add(Dense(1))

model.compile(
    optimizer='adam',
    loss='mean_squared_error'
)

history = model.fit(
    X_train,
    y_train,
    epochs=25,
    batch_size=32,
    validation_data=(X_test, y_test)
)

plt.figure(figsize=(8,4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

predicted_scaled = model.predict(X_test)
predicted_prices = scaler.inverse_transform(predicted_scaled)
actual_prices = scaler.inverse_transform(y_test)

data['MA50'] = data['Close'].rolling(50).mean()
data['MA100'] = data['Close'].rolling(100).mean()

plt.figure(figsize=(10,5))
plt.plot(data['Close'], label='Close')
plt.plot(data['MA50'], label='MA50')
plt.plot(data['MA100'], label='MA100')
plt.legend()
plt.title("Price vs Moving Averages")
plt.show()

plt.figure(figsize=(10,5))
plt.plot(actual_prices, label='Actual Price', color='green')
plt.plot(predicted_prices, label='Predicted Price', color='red')
plt.title("Actual vs Predicted Stock Price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()

model.save("stock_lstm_model.h5")

import joblib
joblib.dump(scaler, "scaler.pkl")

```

5. BACKEND DEVELOPMENT (FASTAPI)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import yfinance as yf
import numpy as np
import pandas as pd
from tensorflow.keras.models import load_model
import joblib

app = FastAPI()

# Allow frontend (Live Server) to talk to backend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def home():
    return {"message": "Backend running"}

# Load trained model and scaler
model =
load_model("model/stock_lstm_model.h5") scaler =
joblib.load("model/scaler.pkl")

@app.get("/predict")
def predict(stock: str):
    df = yf.download(stock, period="120d", progress=False)

    if df.empty:
        return {"error": "Invalid stock symbol"}

    if isinstance(df.columns, pd.MultiIndex):
        df.columns =

    df.columns.get_level_values(0) last_30 =
    df.tail(30)

    close_prices = last_30["Close"].values.reshape(-1,
    1) scaled = scaler.transform(close_prices)

    X = np.array([scaled])
    pred_scaled = model.predict(X, verbose=0)
    next_price = scaler.inverse_transform(pred_scaled)[0][0]

    return {
        "dates":
        last_30.index.strftime("%Y-%m-%d").tolist(), "open":
        last_30["Open"].round(2).tolist(),
        "high": last_30["High"].round(2).tolist(),
        "low": last_30["Low"].round(2).tolist(),
        "close": last_30["Close"].round(2).tolist(),
        "volume": last_30["Volume"].tolist(),
        "next_day_prediction": round(float(next_price),
    2)}
}
```

6. Frontend Code (HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Stock Price Prediction</title>
  <link rel="stylesheet" href="style.css">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>

<div class="container">
  <h1>■ Stock Price Prediction</h1>
  <div class="search-box">
    <input type="text" id="stockInput" placeholder="Enter stock symbol (AAPL, GOOG)">
    <button onclick="getPrediction()">Predict</button>
  </div>

  <!-- Hidden until click -->
  <div id="result" class="hidden">

    <h2>Next Day Predicted Price</h2>
    <p id="nextPrice" class="price"></p>
  </div>
</div>
</body>
</html>
```

```

<h2>Last 30 Days Data</h2>
<table id="dataTable">
<thead>
<tr>
<th>Date</th><th>Open</th><th>High</th><th>Low</th><th>Close</th><th>Volume</th> </tr>
</thead>
<tbody></tbody>
</table>

<h2>Actual vs Predicted (Last 30 Days)</h2>
<canvas id="lineChart"></canvas>
</div>

<script src="script.js"></script>
</body>
</html>

```

Frontend Code (JavaScript)

```

async function getPrediction() {
  const stock = document.getElementById("stockInput").value;
  if (!stock) {
    alert("Enter stock symbol");
    return;
  }

  const res = await fetch(`http://127.0.0.1:8000/predict?stock=${stock}`);
  const data = await res.json();

  if (data.error) {
    alert(data.error);
    return;
  }

  document.getElementById("result").classList.remove("hidden");
  document.getElementById("nextPrice").innerText = "■" + data.next_day_prediction;

  const tbody = document.querySelector("#dataTable tbody");
  tbody.innerHTML = "";
  for (let i = 0; i < data.dates.length; i++) {
    tbody.innerHTML += `
    <tr>
    <td>${data.dates[i]}</td>
    <td>${data.open[i]}</td>
    <td>${data.high[i]}</td>
    <td>${data.low[i]}</td>
    <td>${data.close[i]}</td>
    <td>${data.volume[i]}</td>
    </tr>`;
  }

  const ctx = document.getElementById("lineChart").getContext("2d");
  if (window.chart) window.chart.destroy();

  window.chart = new Chart(ctx, {
    type: "line",
    data: {
      labels: data.dates,
      datasets: [
        {
          label: "Actual Close",
          data: data.close,
          borderColor: "blue",
          fill: false
        },
        {
          label: "Predicted (Next Day)",
          data: [...data.close.slice(1), data.next_day_prediction], borderColor:
"orange",
          borderDash: [5,5],
          fill: false
        }
      ]
    }
  });
}

```

Frontend Code (CSS)

```

body {
  margin: 0;
  padding: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: linear-gradient(120deg, #1f2937, #111827);
  color: #e5e7eb;
}

```



```

}

.container {
width: 85%;
max-width: 1100px;
margin: 40px auto;
background: #ffffff;
color: #1f2937;
padding: 30px;
border-radius: 18px;
box-shadow: 0 20px 40px rgba(0,0,0,0.25);
}

h1 {
text-align: center;
font-size: 32px;
margin-bottom: 25px;
color: #1e3a8a;
}

.search-box {
text-align: center;
margin-bottom: 35px;
}

.search-box input {
padding: 14px;
width: 300px;
border-radius: 10px;
border: 1px solid #c7d2fe;
font-size: 16px;
outline: none;
}

.search-box input:focus {
border-color: #6366f1;
box-shadow: 0 0 8px rgba(99,102,241,0.4);
}

.search-box button {
padding: 14px 26px;
margin-left: 12px;
background: linear-gradient(135deg, #6366f1, #2563eb);
color: white;
border: none;
border-radius: 10px;
cursor: pointer;
font-size: 16px;
transition: transform 0.2s, box-shadow 0.2s;
}

.search-box button:hover {
transform: translateY(-3px);
box-shadow: 0 10px 25px rgba(37,99,235,0.4);
}

.hidden {
display: none;
}

.card {
background: linear-gradient(135deg, #22c55e, #16a34a);
color: white;
padding: 22px;
border-radius: 14px;
margin-bottom: 35px;
box-shadow: 0 12px 30px rgba(34,197,94,0.35);
}

.card h2 {
margin-top: 0;
font-size: 22px;
}

.price {
font-size: 30px;
font-weight: bold;
margin-top: 10px;
}

h2 {
margin-top: 40px;
margin-bottom: 15px;
color: #1e40af;
}

table {
width: 100%;
border-collapse: collapse;
margin-bottom: 35px;

```

```

border-radius: 12px;
overflow: hidden;
}

th {
background: linear-gradient(135deg, #1e3a8a, #2563eb);
color: white;
padding: 14px;
font-size: 15px;
}

td {
padding: 12px;
text-align: center;
font-size: 14px;
color: #1f2937;
}

tr:nth-child(even) {
background: #eef2ff;
}

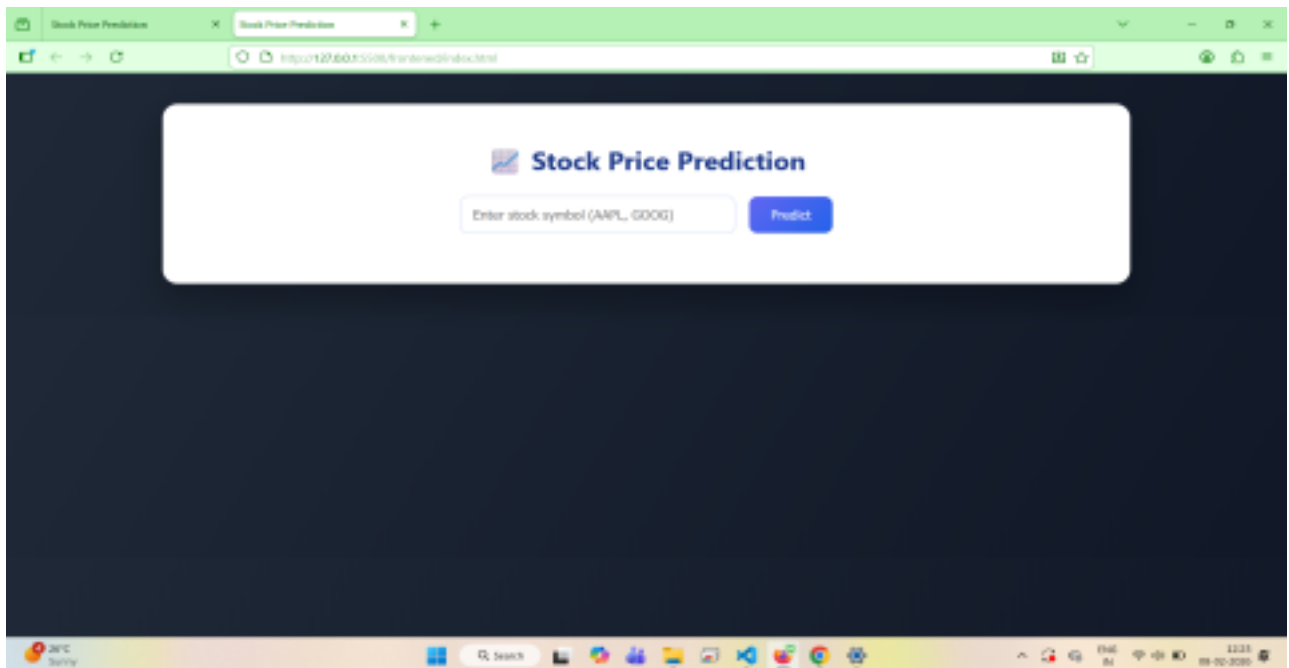
tr:hover {
background: #e0e7ff;
}

canvas {
margin-top: 25px;
background: #f8fafc;
border-radius: 14px;
padding: 12px;
box-shadow: 0 10px 30px rgba(0,0,0,0.15);
}

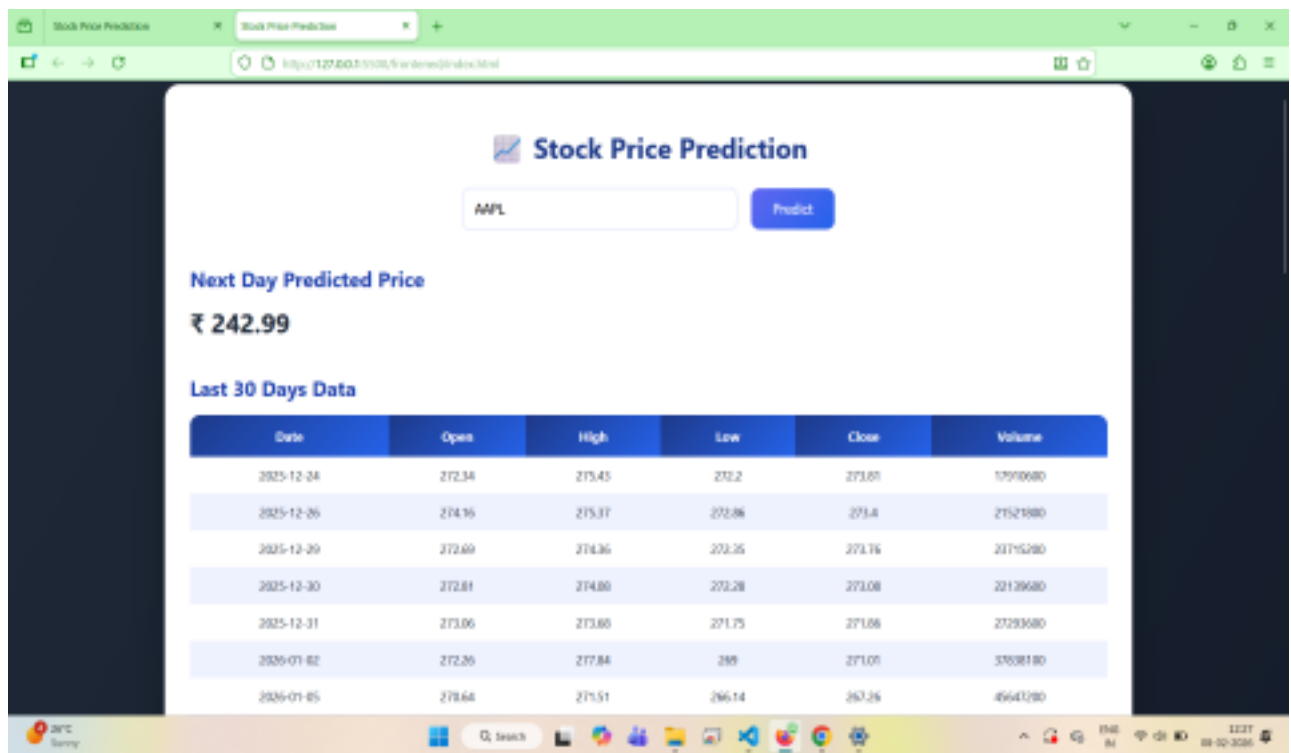
```

7. Outputs / Results

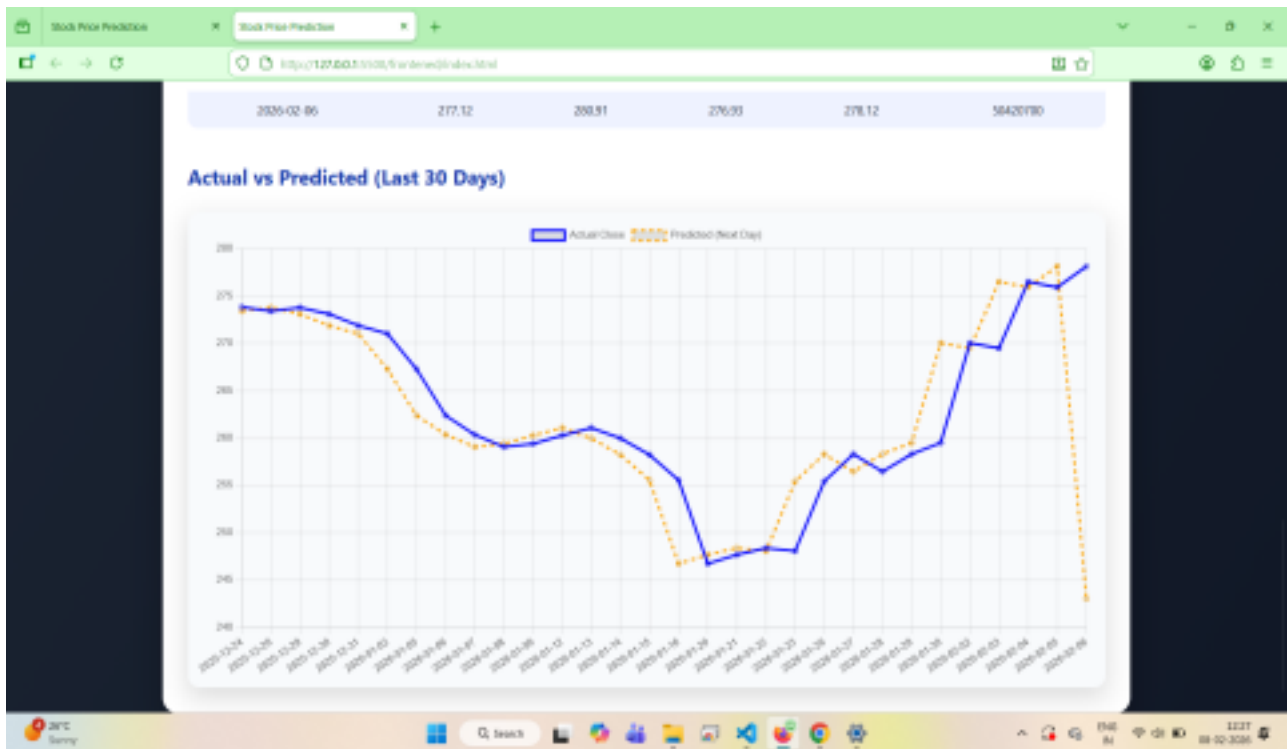
Homepage UI



Prediction Output



Actual vs Predicted Graph



8. Conclusion

This project demonstrates stock price prediction using an LSTM model with full-stack implementation including Google Colab training, FastAPI backend, and web frontend visualization.